Multithreaded Programming

Thread

A thread is the smallest unit of execution inside a program.

A Java program starts with one thread, called the Main Thread

```
public class Demo {
    public static void main(String[] args) {
        System.out.println("Hello");
        System.out.println("Java");
    }
}
```

Here, only one thread executes both statements.

Multithreading

Multithreading is the process of executing two or more threads simultaneously within a single program. It allows multiple tasks to run concurrently.

Advantages

Better CPU utilization
Faster execution
Multiple tasks simultaneously
Responsive applications
Resource sharing

Ways to Create Threads

Method 1: Extending Thread Class

```
class Main extends Thread {
    public void run() {
        System.out.println("Thread is running");
    }
    public static void main(String args[]) {
        Main t = new Main();
        t.start();
    }
}
Thread is running
```

Note: If the class extends the Thread class, the thread can be run by creating an instance of the class and call its start() method.

Method 2: Implementing Runnable Interface

```
class Main implements Runnable {
    public void run() {
        System.out.println("Runnable Thread");
    }
    public static void main(String args[]) {
        Main obj = new Main();
```

```
        Thread t = new Thread(obj);
        t.start();
    }
}
Runnable Thread
```

Note: If the class implements the Runnable interface, the thread can be run by passing an instance of the class to a Thread object's constructor and then calling the thread's start() method.

```
class Pasta extends Thread {
    public void run() {
        System.out.println("Pasta is being prepared.");
    }
}

class Salad extends Thread {
    public void run() {
        System.out.println("Salad is being prepared.");
    }
}

public class Restaurant {
    public static void main(String[] args) {

        Pasta t1 = new Pasta();
        Salad t2 = new Salad();

        t1.start();
        t2.start();
    }
}

Pasta is being prepared.
Salad is being prepared.
(The order may change because both threads run concurrently.)
```

```
class Cooking extends Thread {
    String food;
    Cooking(String food) {
        this.food = food;
    }
    public void run() {
        System.out.println(food + " is being prepared.");
    }
}

public class Restaurant {
    public static void main(String[] args) {
```

```
        Cooking t1 = new Cooking("Pasta");
        Cooking t2 = new Cooking("Salad");
        Cooking t3 = new Cooking("Rice");

        t1.start();
        t2.start();
        t3.start();
    }
}
```

Rice is being prepared.
Pasta is being prepared.
Salad is being prepared.
(Output order is not fixed.)

```
class Cooking extends Thread {
    String food;
    Cooking(String food) {
        this.food = food;
    }
    public void run() {
        System.out.println(food + " prepared by " + getName());
    }
}
public class Restaurant {
    public static void main(String[] args) {

        Cooking t1 = new Cooking("Pasta");
        Cooking t2 = new Cooking("Salad");

        t1.start();
        t2.start();
    }
}
```

Pasta prepared by Thread-0
Salad prepared by Thread-1

Differences between "extending" and "implementing" Threads

The major difference is that when a class extends the Thread class, you cannot extend any other class, but by implementing the Runnable interface, it is possible to extend from another class as well.

Case 1: Extending Thread (Cannot Extend another Class)

```
class Animal {
    void sound() {
        System.out.println("Animal Sound");
    }
}
class Dog extends Animal, Thread { // “NOT ALLOWED IN JAVA”
```

```
    public void run() {
        System.out.println("Thread Running");
    }
}
```
Compilation Error

Case 2: Using Runnable (Allowed)

```
class Animal {
    void sound() {
        System.out.println("Animal Sound");
    }
}
class Dog extends Animal implements Runnable { // ALLOWED
    public void run() {
        System.out.println("Thread Running");
    }
    public static void main(String args[]) {
        Dog d = new Dog();
        Thread t = new Thread(d);
        d.sound();
        t.start();
    }
}
```
Animal Sound
Thread Running

Concurrency Problems

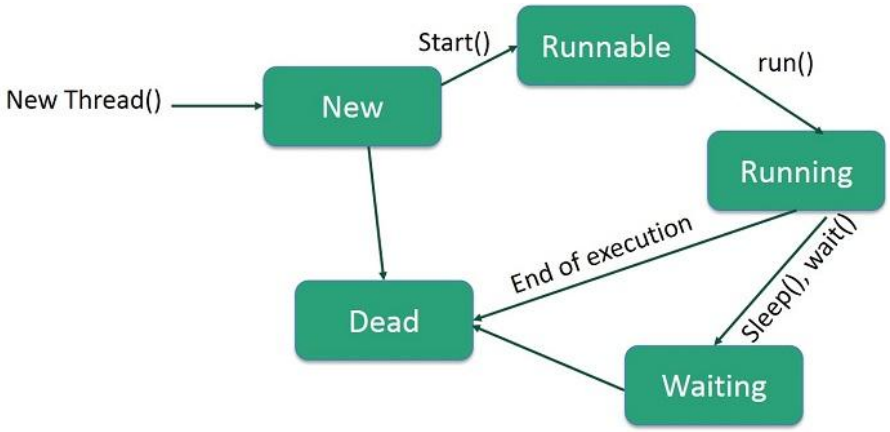
```
public class Main extends Thread {
  public static int amount = 10;
  public static void main(String[] args) {
    Main obj = new Main();
    obj.start();
    System.out.println(amount);
    amount++;
    System.out.println(amount);
  }
  public void run() {
    amount++;
  }
}
```
10
11
OR
10
12
OR
11
12

Use isAlive() to prevent concurrency problems

```
public class Main extends Thread {
  public static int amount = 0;

  public static void main(String[] args) {
    Main obj = new Main();
    obj.start();
    // Wait for the thread to finish
    while(obj.isAlive()) {
      System.out.println("Waiting...");
    }
    System.out.println("Main: " + amount);
    amount++;
    System.out.println("Main: " + amount);
  }
  public void run() {
    amount++;
  }
}
Waiting...
Waiting...
Main: 1
Main: 2
OR
Waiting...
Main: 1
Main: 2
```

Life Cycle of a Thread



- **New** − A new thread begins its life cycle in the new state. It remains in this state until the program starts the thread. It is also referred to as a **born thread**.

- **Runnable** − after a newly born thread is started, the thread becomes runnable. A thread in this state is considered to be executing its task.
- **Waiting** − Sometimes, a thread transitions to the waiting state while the thread waits for another thread to perform a task. A thread transitions back to the runnable state only when another thread signals the waiting thread to continue executing.
- **Timed Waiting** − A runnable thread can enter the timed waiting state for a specified interval of time. A thread in this state transition back to the runnable state when that **time interval** expires or when the event it is waiting for occurs.
- **Terminated (Dead)** − A runnable thread enters the terminated state when it completes its task or otherwise terminates.

```
class MyThread extends Thread {
    public void run() {
        System.out.println("Running State");
        try {
            System.out.println("Sleeping...");
            Thread.sleep(3000);      // Waiting State
        }
        catch(Exception e) { } //InterruptedException (checked exception)
        System.out.println("Thread Finished");
    }
}
public class Main {
    public static void main(String[] args) {
        MyThread t = new MyThread();
        // NEW State
        System.out.println("1. New State");
        // Runnable State
        t.start();
        System.out.println("2. Runnable State");
        while(t.isAlive()) {
            // Wait until thread finishes. Not a thread state
        }
        // Dead State
        System.out.println("3. Dead State");
    }
}
1. New State
2. Runnable State
Running State
Sleeping...
Thread Finished
3. Dead State
```

OR

1. New State
Running State
Sleeping...
2. Runnable State
Thread Finished
3. Dead State

Thread Priorities

Java thread priorities are in the range between MIN_PRIORITY (a constant of 1) and MAX_PRIORITY (a constant of 10). By default, every thread is given priority NORM_PRIORITY (a constant of 5).

Threads with higher priority are more important to a program and should be allocated processor time before lower-priority threads.

However, thread priorities cannot guarantee the order in which threads execute and are very much platform dependent.

```
public class Main extends Thread {
      public void run() {
       System.out.println(getName() + " executing with Priority: " + getPriority());
   }

   public static void main(String[] args) {
       Main lowPriorityThread = new Main();
       Main midPriorityThread = new Main();
       Main highPriorityThread = new Main();

       // Set thread priorities
       lowPriorityThread.setPriority(Thread.MIN_PRIORITY); // Priority 1
       midPriorityThread.setPriority(Thread.NORM_PRIORITY); // Priority 5
       highPriorityThread.setPriority(Thread.MAX_PRIORITY); // Priority 10

       // Start the threads
       lowPriorityThread.start();
       midPriorityThread.start();
       highPriorityThread.start();
   }
}
Thread-0 executing with Priority: 1
Thread-1 executing with Priority: 5
Thread-2 executing with Priority: 10

OR

Thread-0 executing with Priority: 1
Thread-2 executing with Priority: 10
```

```
Thread-1 executing with Priority: 5
```

Synchronizing Threads

Thread synchronization means controlling multiple threads so that only one thread accesses a shared resource at a time.

It is mainly used to avoid data inconsistency when multiple threads modify the same data.

Problem without Synchronization

```
class Account {
    int balance = 1000;
    void withdraw(int amount) {
        if (balance >= amount) {
            System.out.println(Thread.currentThread().getName()
                    + " is withdrawing " + amount);
            balance = balance - amount;
        }
    }
}
public class Main {
    public static void main(String[] args) {
        Account a = new Account();
        Thread t1 = new Thread(() -> a.withdraw(700), "Thread 1");
        Thread t2 = new Thread(() -> a.withdraw(700), "Thread 2");
        t1.start();
        t2.start();
    }
}
Thread-0 is withdrawing 700
Thread-1 is withdrawing 700
OR

Thread-1 is withdrawing 700
Thread-0 is withdrawing 700
```

Using synchronized

```
class Account {
    int balance = 1000;
    synchronized void withdraw(int amount) {
        if (balance >= amount) {
            System.out.println(Thread.currentThread().getName()
                    + " is withdrawing " + amount);
            balance = balance - amount;
            System.out.println("Remaining balance = " + balance);
        }
        else {
            System.out.println("Insufficient balance");
        }
    }
}
```

```
public class Main {
    public static void main(String[] args) {
        Account a = new Account();
        Thread t1 = new Thread(() -> a.withdraw(700), "Thread 1");
        Thread t2 = new Thread(() -> a.withdraw(700), "Thread 2");
        t1.start();
        t2.start();
    }
}
Thread-0 is withdrawing 700
Remaining balance = 300
Insufficient balance
OR

Thread-1 is withdrawing 700
Remaining balance = 300
Insufficient balance
```

Mutual Exclusion

Only one thread can enter the critical section at a time.

Type	Meaning
Synchronized Method	Lock the whole method for one object
Synchronized Block	Lock only a particular block
Static Synchronization	Use one common class-level lock

1. Mutual Exclusion using Synchronized Method

The simplest form is to put synchronized before the method.

```
class Test {
    synchronized void display() {
        for (int i = 1; i <= 5; i++) {
            System.out.println(
                Thread.currentThread().getName() + " : " + i
            );
        }
    }
}
public class Main {
    public static void main(String[] args) {
        Test obj = new Test();
        Thread t1 = new Thread(() -> obj.display(), "Thread 1");
        Thread t2 = new Thread(() -> obj.display(), "Thread 2");
        t1.start();
        t2.start();
    }
}
Possible Output
Thread-0 : 1
```

```
Thread-0 : 2
Thread-0 : 3
Thread-0 : 4
Thread-0 : 5
Thread-1 : 1
Thread-1 : 2
Thread-1 : 3
Thread-1 : 4
Thread-1 : 5
OR
Thread 1 : 1
Thread 1 : 2
Thread 1 : 3
Thread 1 : 4
Thread 1 : 5
Thread-0 : 1
Thread-0 : 2
Thread-0 : 3
Thread-0 : 4
Thread-0 : 5
```

2. Synchronized Block

```
class Table {
    void printTable(int n) {
        synchronized (this) {
            for (int i = 1; i <= 5; i++) {
                System.out.println(
                    Thread.currentThread().getName() + " : " + (n * i)
                );
            }
        }
    }
}

public class Main {
    public static void main(String[] args) {
        Table t = new Table();

        Thread t1 = new Thread(() -> t.printTable(5));
        Thread t2 = new Thread(() -> t.printTable(10));

        t1.start();
        t2.start();
    }
}
Possible Output
Thread-0 : 5
Thread-0 : 10
Thread-0 : 15
Thread-0 : 20
```

```
Thread-0 : 25
Thread-1 : 10
Thread-1 : 20
Thread-1 : 30
Thread-1 : 40
Thread-1 : 50
OR
Thread-1 : 10
Thread-1 : 20
Thread-1 : 30
Thread-1 : 40
Thread-1 : 50
Thread-0 : 5
Thread-0 : 10
Thread-0 : 15
Thread-0 : 20
Thread-0 : 25
```

3. Static Synchronization

When a static method is synchronized, the class itself is locked.

```
class Table {
    static synchronized void printTable(int n) {
        for (int i = 1; i <= 5; i++) {
            System.out.println(
                Thread.currentThread().getName() + " : " + (n * i)
            );
        }
    }
}

public class Main {
    public static void main(String[] args) {

        Thread t1 = new Thread(() -> Table.printTable(5));
        Thread t2 = new Thread(() -> Table.printTable(10));

        t1.start();
        t2.start();
    }
}
Possible Output
Thread-0 : 5
Thread-0 : 10
Thread-0 : 15
Thread-0 : 20
Thread-0 : 25
Thread-1 : 10
Thread-1 : 20
Thread-1 : 30
Thread-1 : 40
```

```
Thread-1 : 50
OR
Thread-1 : 10
Thread-1 : 20
Thread-1 : 30
Thread-1 : 40
Thread-1 : 50
Thread-0 : 5
Thread-0 : 10
Thread-0 : 15
Thread-0 : 20
Thread-0 : 25
```

```
class Account {
    static synchronized void withdraw() {
        // locked by Thread 1
    }
    static synchronized void deposit() {
        // locked by Thread 1
    }

    static void display() {
        // NOT synchronized
    }
}
```

```
Thread 1 → Account.withdraw()    Running
Thread 2 → Account.withdraw()    Waiting
Thread 3 → Account.deposit()     Waiting
Thread 4 → Account.display()     Running
```

Type	Syntax	What is locked?	Can another thread access?
Synchronized Method	synchronized void withdraw()	Particular object	Another thread can use a different object
Synchronized Block	synchronized(this) { }	Particular block inside a method	Code outside the block can run
Static Synchronization	static synchronized void withdraw()	Class-level lock	Other threads using the same class lock must wait

Inter-Thread Communication

It allows one thread to communicate with another thread and coordinate their execution.

Method	Meaning
join	One thread has to wait for another thread to finish
wait()	Makes the current thread wait
notify()	Wakes up **one** waiting thread

notifyAll()	Wakes up **all** waiting threads

```
class MyThread extends Thread {
    public void run() {
        for (int i = 1; i <= 3; i++) {
            System.out.println("Child Thread: " + i);
        }
    }
}
public class Main {
    public static void main(String[] args) throws InterruptedException {
        MyThread t = new MyThread();
        t.start();
        t.join();   // Main thread, wait until thread t finishes
        System.out.println("Main Thread finished");
    }
}
Child Thread: 1
Child Thread: 2
Child Thread: 3
Main Thread finished
```

```
class Test {
    synchronized void display() {
        try {
            System.out.println("Thread 1 is waiting...");
            wait();
            System.out.println("Thread 1 is running again");
        }
        catch (Exception e) {
            System.out.println(e);
        }
    }
    synchronized void wakeUp() {
        System.out.println("Thread 2 is running");
        notify();
    }
}
public class Main {
    public static void main(String[] args) {
        Test obj = new Test();
        Thread t1 = new Thread(() -> obj.display());
        Thread t2 = new Thread(() -> obj.wakeUp());
        t1.start();
        t2.start();
    }
}
```

```
Possible output
Thread 1 is waiting...
Thread 2 is running
Thread 1 is running again
OR
Thread 2 is running
Thread 1 is waiting...
```

```
class Test {
    synchronized void display() {
        try {
            System.out.println(Thread.currentThread().getName()
                    + " is waiting...");
            wait();
            System.out.println(Thread.currentThread().getName()
                    + " is running again");
        }
        catch (Exception e) {
            System.out.println(e);
        }
    }
    synchronized void wakeUp() {
        System.out.println("Thread 2 is running");
        notifyAll();
    }
}

public class Main {
    public static void main(String[] args) {
        Test obj = new Test();
        Thread t1 = new Thread(() -> obj.display(), "Thread 1");
        Thread t2 = new Thread(() -> obj.display(), "Thread 3");
        Thread t3 = new Thread(() -> obj.wakeUp(), "Thread 2");
        t1.start();
        t2.start();
        t3.start();
    }
}
Possible output
Thread 1 is waiting...
Thread 3 is waiting...
Thread 2 is running
Thread 1 is running again
Thread 3 is running again
OR
Thread 3 is waiting...
Thread 1 is waiting...
Thread 2 is running
Thread 3 is running again
```

Thread 1 is running again
OR
Thread 2 is running
Thread 1 is waiting...
Thread 3 is waiting...

Method	Purpose
suspendThread()	Temporarily pauses the thread
resumeThread()	Allows the suspended thread to continue
stopThread()	Tells the thread to terminate